

Representation of statements in Predicate logic

1. Marcus was a man.

$$man(Marcus)$$

2. All Pompeians died when the volcano erupted in 79 A.D.

$$erupted(volcano, 79) \wedge \forall x : [Pompeian(x) \rightarrow died(x, 79)]$$

3. No mortal lives longer than 150 years.

$$\forall x : \forall t_1 : \forall t_2 : mortal(x) \wedge born(x, t_1) \wedge gt(t_2 - t_1, 150) \rightarrow dead(x, t_2)$$

4. It is 2025 now.

$$now = 2025$$

5. Alive means not dead.

$$\forall x : \forall t : [alive(x, t) \rightarrow \neg dead(x, t)] \wedge [\neg dead(x, t) \rightarrow alive(x, t)]$$

6. If someone dies, then he is dead at all later times.

$$\forall x : \forall t_1 : \forall t_2 : died(x, t_1) \wedge gt(t_2, t_1) \rightarrow dead(x, t_2)$$

7. All Romans were either loyal to Caesar or hated him.

$$\forall x : Roman(x) \rightarrow loyalto(x, Caesar) \vee hate(x, Caesar)$$

Exclusive representation:

$$\forall x : Roman(x) \rightarrow [(loyalto(x, Caesar) \vee hate(x, Caesar)) \wedge \neg (loyalto(x, Caesar) \wedge hate(x, Caesar))]$$

8. **Everyone** is loyal to someone.

$$\forall x : \exists y : loyalto(x, y)$$

9. People only try to assassinate rulers they are not loyal to.

$$\forall x : \forall y : (person(x) \wedge ruler(y) \wedge tryassassinate(x, y) \rightarrow \neg loyalto(y, x))$$

Algorithm: Propositional Resolution

1. Convert all the propositions of F to clause form.
2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in step 1.
3. Repeat until either a contradiction is found or no progress can be made:
 - (a) Select two clauses. Call these the *parent clauses*.
 - (b) Resolve them together. The resulting clause, called the *resolvent*, will be the disjunction of all the literals of both of the parent clauses with the following **exception**: If there are any pairs of literals L and $\neg L$ such that one of the parent clauses contains L and the other contains $\neg L$, then select one such pair and eliminate both L and $\neg L$ from the resolvent.
 - (c) If the resolvent is the empty clause, then a contradiction has been found. If it is not, then add it to the set of clauses available to the procedure.

Algorithm: Unify($L1, L2$)

1. If $L1$ or $L2$ are both variables or constants, then:
 - (a) If $L1$ and $L2$ are identical, then return **NIL**.
 - (b) Else if $L1$ is a variable, then if $L1$ occurs in $L2$, then return **{FAIL}**, else return $(L2/L1)$.
 - (c) Else if $L2$ is a variable, then if $L2$ occurs in $L1$, then return **{FAIL}**, else return $(L1/L2)$.
 - (d) Else return **{FAIL}**.
2. If the initial predicate symbols in $L1$ and $L2$ are not identical, then return **{FAIL}**.
3. If $L1$ and $L2$ have a different number of arguments, then return **{FAIL}**.
4. Set $SUBST$ to **NIL**. (At the end of this procedure, $SUBST$ will contain all the substitutions used to unify $L1$ and $L2$.)
5. For $i \leftarrow 1$ to number of arguments in $L1$:
 - (a) Call **Unify** with the i th argument of $L1$ and the i th argument of $L2$, putting result in S .
 - (b) If S contains **FAIL**, then return **{FAIL}**.
 - (c) If S is not equal to **NIL**, then:
 - (i) Apply S to the remainder of both $L1$ and $L2$.
 - (ii) $SUBST := \mathbf{APPEND}(S, SUBST)$.
6. Return $SUBST$.

Algorithm: Resolution in Predicate Logic

1. Convert all the statements of F to clause form.
2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in step 1.
3. Repeat until either a contradiction is found, no progress can be made, or a predetermined amount of effort has been expended.
 - (a) Select two clauses. Call these the *parent clauses*.
 - (b) Resolve them together. The resolvent will be the disjunction of all the literals of both parent clauses with appropriate substitutions performed and with the following exception: If there is one pair of literals T_1 and $\neg T_2$ such that one of the parent clauses contains T_2 and the other contains T_1 , and if T_1 and T_2 are unifiable, then neither T_1 nor T_2 should appear in the resolvent. We call T_1 and T_2 *complementary literals*. Use the substitution produced by the unification to create the resolvent. If there is more than one pair of complementary literals, only one pair should be omitted from the resolvent.
 - (c) If the resolvent is the empty clause, then a contradiction has been found. If it is not, then add it to the set of clauses available to the procedure.